

Linguagem SQL para Controle de Acesso (DCL)

Quem acessa, o que pode fazer e como proteger os dados do banco

"Conceder, limitar e revogar: o trio que mantém seu banco de dados seguro."

O SQL é dividido em famílias — cada uma tem uma missão

DDL

DATA DEFINITION

O que controla: as **estruturas** das tabelas do banco de dados.

CREATE

DROP

ALTER

Responde a: "Como as tabelas são criadas e modificadas?"

DML

DATA MANIPULATION

O que controla: os **dados armazenados** dentro das tabelas.

INSERT

UPDATE

DELETE

Responde a: "O que entra, muda e sai das tabelas?"

DCL

DATA CONTROL

O que controla: as **permissões de acesso** — quem pode fazer o quê.

GRANT

REVOKE

Responde a: "Quem pode acessar o banco e o que cada usuário pode fazer?"

DDL molda o banco, **DML** move os dados, **DCL** protege tudo isso.

MOTIVAÇÃO

Sem DCL, qualquer pessoa conectada ao banco pode causar danos irreversíveis

RISCOS DE NÃO CONTROLAR O ACESSO

01



Excluir estruturas vitais

Apagar tabelas essenciais do sistema com comandos destrutivos como `DROP TABLE`.

02



Corromper registros

Alterar dados de vendas, estoques ou transações com `UPDATE` malicioso ou acidental.

03



Vazar dados sensíveis

Visualizar salários, CPFs e informações pessoais de terceiros com `SELECT` sem controle.

PAPEL 01

Administrador do Banco (DBA)

Possui **acesso total**: cria usuários, concede e revoga permissões.

PAPEL 02

Usuários finais e aplicações

Desenvolvedores, analistas de dados e sistemas web com acessos **restritos e específicos** para suas tarefas.

O controle de acesso também garante **conformidade com leis de proteção de dados (LGPD)**: limitar quem vê dados pessoais não é burocracia — é a primeira linha de defesa.

Privilégios são os “direitos” que a DCL distribui — em dois níveis



Privilégio: o direito de executar um comando SQL específico ou de acessar um determinado objeto no banco de dados.

NÍVEL 1

Privilégios do Sistema

Permitem realizar **ações estruturais** no banco de dados como um todo.

CREATE SESSION

Conectar ao banco de dados

CREATE TABLE

Criar novas tabelas

CREATE USER

Criar novos usuários

NÍVEL 2

Privilégios de Objeto

Permitem executar **ações diretas** sobre tabelas, views ou procedures específicas.

SELECT

Consultar dados

INSERT

Inserir registros

UPDATE

Atualizar registros

DELETE

Excluir registros

EXECUTE

Executar procedures

Na prática diária de análise de dados, os **privilégios de objeto** (SELECT, INSERT, UPDATE, DELETE) são os mais utilizados.

Agrupe permissões em um perfil em vez de repetir comandos para cada usuário

SOLUÇÃO EM 3 PASSOS

PROBLEMA

Conceder permissão por permissão para cada funcionário

Gerar 10 comandos individuais para cada novo analista é um processo **ineficiente e propenso a erros**: a cada contratação, o DBA repete o trabalho inteiro.

DEFINIÇÃO

Um perfil (role) é um **conjunto nomeado de privilégios agrupados**, que pode ser associado a quantos usuários forem necessários.

1 Criar um perfil nomeado

Ex.: `CREATE ROLE 'analista_dados'` — um nome que representa o cargo.



2 Atribuir as permissões ao perfil

As 10 permissões são gravadas **uma única vez** no perfil, nunca no usuário.



3 Associar o usuário ao perfil

Novo funcionário entra? Basta uma linha: `GRANT 'analista_dados' TO novo_usuario .`



Centralização

A política de acesso vive em um único lugar.



Manutenção fácil

50 novos funcionários = 50

associações, não 500 GRANTS.



Consistência

Todo usuário do mesmo cargo tem exatamente as mesmas permissões.

GRANT: conceda permissões — palavra a palavra

SINTAXE BASE — PRIVILÉGIOS DE OBJETO

```
GRANT privilegio ON nome_tabela TO  
usuario;
```

Tradução: “conceda este privilégio, sobre esta tabela, a este usuário.”

EXEMPLO PRÁTICO

```
GRANT SELECT, INSERT ON vendas TO  
analista_joao;
```

Analista_joao agora pode consultar **E** inserir dados na tabela **vendas** — e nada mais.

⚠ ACESSO TOTAL — CUIDADO

```
GRANT ALL PRIVILEGES ON clientes TO  
gerente_maria;
```

Use apenas para quem **realmente precisa de tudo** — lembre-se do princípio do menor privilégio.

Regras de sintaxe

GRANT → ON → TO

- Separe vários privilégios com vírgula: `GRANT SELECT, INSERT, UPDATE ...`
- O GRANT também concede **perfis (roles)** a usuários e outros perfis.

REVOKE: remova permissões com precisão cirúrgica

SINTAXE BASE — PRIVILÉGIOS DE OBJETO

```
REVOKE privilegio ON nome_tabela FROM usuario
```

Atenção: aqui se usa **FROM** — no GRANT usamos TO. É o espelho exato do comando que concede.

EXEMPLO PRÁTICO

```
REVOKE INSERT ON vendas FROM analista_joao
```

Tradução: o analista_joao **mantém o SELECT**, mas **perde o direito de inserir** novos registros na tabela vendas.

 Use **FROM** para indicar de quem as permissões estão sendo retiradas — a sintaxe é espelhada em relação ao GRANT.

PERGUNTA DE FIXAÇÃO

“REVOKE apaga o usuário?”

Não. Ele apenas remove permissões e privilégios de acesso. O usuário continua existindo no sistema e ainda pode se conectar — só não consegue executar as ações revogadas. Para apagar um usuário, use o comando:

```
DROP USER
```

O ESPELHO GRANT ↔ REVOKE

GRANT ... TO

Entrega o privilégio **ao** usuário.

REVOKE ... FROM

Retira o privilégio **de** quem o possuía.

← mesma estrutura, direção invertida →

Passos 1 e 2: criar usuários e definir permissões diferenciadas

PASSO 1

Criar os usuários (cargos fictícios)

O professor simula **dois funcionários com cargos diferentes** em uma empresa, criando contas de acesso com senhas no SGBD.

```
CREATE USER
'analista_junior'@'localhost'
IDENTIFIED BY 'Senha123!'; CREATE
USER
'estagiario_dados'@'localhost'
IDENTIFIED BY 'Senha456!';
```

 Por enquanto, os usuários existem no sistema — mas **não podem fazer nada**, pois ainda não receberam permissões.

PASSO 2

Definir permissões na tabela financeira

Permissões distintas sobre a tabela **relatorio_financeiro**, refletindo a realidade de cada cargo.

analista_junior

Precisa **ler e inserir** novos dados: recebe SELECT + INSERT.

```
GRANT SELECT, INSERT ON
empresa_db.relatorio_financeiro
TO
'analista_junior'@'localhost';
```

estagiario_dados

Só **visualiza** dados para gerar dashboards: recebe apenas SELECT.

```
GRANT SELECT ON
empresa_db.relatorio_financeiro
TO
'estagiario_dados'@'localhost';
```



Insight da aula: analista opera (lê e insere); estagiário só visualiza. A DCL permite que a permissão espelhe exatamente o papel de cada pessoa na empresa.

Passos 3 e 4: revogar acessos e agrupar permissões em um perfil

PASSO 3 Revogar acessos

O professor simula uma **mudança de escopo** na empresa: o analista júnior não pode mais inserir dados diretamente na tabela financeira.

🕒 *Cenário: política interna mudou — a inserção em dados financeiros agora passa por aprovação de outro setor.*

```
REVOKE INSERT ON  
empresa_db.relatorio_financeiro FROM  
'analista_junior'@'localhost';
```

💡 *Tradução: o analista_junior **continua lendo** o relatório, mas **perde o direito de inserir** registros — de forma imediata e apenas neste privilégio.*

💡 *O REVOKE é **preciso e cirúrgico**: remove uma permissão específica sem afetar as demais do usuário.*

PASSO 4 Criar e atribuir um perfil (Role)

Demonstração do **agrupamento de permissões**: um perfil de leitura reutilizável para vários usuários.

```
CREATE ROLE 'perfil_leitura'; GRANT SELECT ON  
empresa_db.vendas TO 'perfil_leitura'; GRANT SELECT ON  
empresa_db.produtos TO 'perfil_leitura'; GRANT  
'perfil_leitura' TO 'estagiario_dados'@'localhost';
```

1 · CRIAR O
PERFIL



2 · DAR PERMISSÕES AO
PERFIL



3 · ASSOCIAR OS
USUÁRIOS

💡 ***Insight da aula:** se amanhã chegar um novo estagiário, basta executar a última linha — **zero retrabalho**, sem repetir GRANTS individuais.*

Vocês são os DBAs de uma rede de farmácias



Cenário: estruture a política de acesso do banco de dados da rede com as tabelas `medicamentos`, `vendas` e `funcionarios` — aplicando cada conceito da aula.

ROTEIRO DA ATIVIDADE

1

Criar dois usuários

Use `CREATE USER` para `vendedor_loja` e `auditor_fiscal`.

3

Conceder ao perfil

`SELECT` sobre `medicamentos` e `INSERT` sobre `vendas` para o `nivel_operacional`.

5

Conceder diretamente ao auditor

`SELECT` nas três tabelas (`medicamentos`, `vendas`, `funcionarios`) para o `auditor_fiscal`.

7

Testar no SGBD

Execute os comandos e anote os erros (`Access Denied`) quando um usuário operar **sem o GRANT devido**.

2

Criar um perfil de acesso

Use `CREATE ROLE` para o perfil `nivel_operacional`.

4

Associar usuário ao perfil

Use `GRANT 'nivel_operacional' TO` para vincular o `vendedor_loja`.

6

Simular o incidente

O vendedor tentou `UPDATE` de preço sem autorização: escreva o **REVOKE** que retira essa permissão imediatamente.



Dica didática: cada etapa usa exatamente um conceito da aula — usuário → perfil → GRANT no perfil → GRANT ao usuário → REVOKE.

Três pontos-chave validados na tela: sintaxe, inspeção e um perigo a evitar

01

PONTO 1

Validação de sintaxe

O professor revisa na tela os comandos `GRANT` e `REVOKE` escritos pelas duplas, conferindo a **estrutura completa**: vírgulas entre privilégios, cláusulas `ON` / `TO` / `FROM` e nomes de usuários entre aspas.

- ✓ Um comando **mal escrito não concede nem revoga** nada — ele simplesmente falha.

02

PONTO 2

Inspeção das permissões ativas

Como conferir o que cada usuário realmente tem? Use os **comandos de inspeção do SGBD** antes de qualquer auditoria:

```
SHOW GRANTS FOR  
'vendedor_loja'@'localhost';
```

- 🔍 Tradução: **"mostre todas as permissões ativas deste usuário"** — a ferramenta oficial de verificação.

03

PONTO 3 · ALERTA

O perigo do WITH GRANT OPTION

Essa cláusula permite que um **usuário comum repasse suas permissões** para outros, sem autorização do DBA:

```
GRANT SELECT ON medicamentos TO  
'vendedor_loja'@'localhost' WITH  
GRANT OPTION;
```

- ⚠ Risco: **quebra total do controle centralizado** — o privilégio se espalha fora do alcance do DBA.



Regra de ouro: nunca conceda **WITH GRANT OPTION** a usuários comuns — apenas o DBA deve distribuir permissões.

5 perguntas que provam o domínio da DCL

Q1 Qual a diferença entre DML, DDL e DCL?

DDL define estruturas; DML manipula dados; DCL controla **autorizações e permissões** de acesso aos objetos e dados.

Q2 E se um usuário sem permissão executar um DELETE?

O SGBD **bloqueia na hora** e retorna erro de acesso negado (`Access Denied / Insufficient Privileges`), mantendo a base intacta.

Q3 Vantagem de usar perfis (Roles)?

Centralização e manutenção fácil. Mudou a política ou chegaram 50 funcionários? Altere o perfil — sem dezenas de `GRANT` s individuais.

Q4 REVOKE apaga o usuário?

Não. Só remove permissões — o usuário continua existindo e pode se conectar. Para apagar, use `DROP USER` .

Q5 O que é o princípio do menor privilégio e como ele se aplica à DCL?

Cada usuário deve ter **apenas as permissões estritamente necessárias** para o seu trabalho diário. Use `GRANT` específico (ex.: `SELECT` em uma tabela) e **evite** `GRANT ALL PRIVILEGES` para usuários comuns.

SÍNTESE FINAL



Segurança de dados começa com DCL: crie perfis, conceda o mínimo necessário e audite sempre com SHOW GRANTS.